

NBA Free Throw Prediction Using Machine Learning Classification Models

Habibatallah Mahdi, Sama Abdelsadek, Mohamed Allam, Shaza Hossam, Ahmed Shaban

Department of Computer Science

School of Information Technology and Computer Science

Nile University

El Sheikh Zayed, Giza, Egypt

Abstract—Free-throw shooting is a decisive, high-frequency skill in basketball, yet it is only partially explained by the rest of a player’s box-score profile. This project presents a machine learning system that classifies an NBA player-season as a *good* free-throw shooter (free-throw percentage at or above 75%) or a *below-average* free-throw shooter (below 75%) using season-long statistics. A dataset of 26,292 player-season records covering the 1975–2025 NBA seasons was collected by web scraping Basketball Reference and was augmented with scraped player height and weight from a second source. After removing traded-player duplicates and keeping only player-seasons with a meaningful free-throw sample, 15,020 records described by 39 features remained. Particular care was taken to avoid data leakage: free throws made, free-throw attempts, and free-throw percentage are excluded from the feature set because they directly determine the target label. Five classification algorithms — K-Nearest Neighbors, Support Vector Classifier, Logistic Regression, Random Forest, and Gradient Boosting — were trained with default-configuration estimators and compared using accuracy, precision, recall, F1-score, and confusion matrices. Gradient Boosting achieved the highest test accuracy of 78.06%. The system is deployed as an interactive Streamlit web application that generates real-time predictions for user-supplied player profiles.

Index Terms—Machine Learning, Sports Analytics, NBA, Classification, Free Throw Prediction, Data Leakage, Web Scraping

I. INTRODUCTION

Machine learning has become one of the most influential technologies in sports analytics. Basketball organizations increasingly use data-driven systems to evaluate players, optimize team strategies, and support recruitment and rotation decisions. Among individual skills, free-throw shooting stands out: free throws are uncontested, occur in large volume, and frequently decide the outcome of close games during clutch moments.

Free-throw efficiency is, however, a specific physical and mental skill that is only partially correlated with the rest of a player’s statistical profile. A high-scoring guard and a low-usage center may share many box-score similarities yet differ sharply at the free-throw line. This makes free-throw classification a genuinely non-trivial learning problem rather than a simple statistical lookup.

This project builds a machine learning classification system that predicts whether an NBA player-season is a good or

a below-average free-throw shooter, using season-long box-score statistics together with engineered and historical features. The project is organized into two phases:

- **Phase 1:** Problem definition, data collection, preprocessing, feature engineering, and initial model training.
- **Phase 2:** Systematic model evaluation and comparison, result visualization, deployment as an interactive web application, and preparation of this final paper.

The main contributions of the work are: (1) an original dataset assembled by web scraping two distinct Basketball Reference sources and integrating them; (2) a preprocessing pipeline that explicitly prevents data leakage, which is the single most common cause of misleadingly optimistic results in this type of task; (3) a fair comparison of five classification algorithms trained with default-configuration estimators; and (4) a deployed Streamlit dashboard that makes the trained models usable in real time.

II. PROBLEM STATEMENT

The objective of this project is to classify each NBA player-season into one of two categories based on free-throw percentage (FT%):

- **Good free-throw shooter** — $FT\% \geq 0.75$
- **Below-average free-throw shooter** — $FT\% < 0.75$

The task is therefore a supervised binary classification problem. The challenge is to learn this distinction *without* access to any free-throw quantity for the season being predicted, relying instead on the player’s other statistics, physical attributes, engineered ratios, and free-throw record from *previous* seasons.

A correctly designed system of this type can support coaches, analysts, scouts, and front-office staff by flagging players who are likely to be reliable or unreliable at the line, informing rotation choices, late-game lineups, and contract evaluation.

III. DATASET DESCRIPTION

A. Data Source

In accordance with the course requirement that datasets be self-collected from reputable sources rather than downloaded ready-made, the dataset was assembled by web scraping Basketball Reference using Python with the `requests`,

BeautifulSoup, and lxml libraries. Two distinct sources were scraped and merged:

- Per-game player statistics for every NBA season from 1975 to 2025, from the per-game season leaderboards.¹
- Player biographical data (height and weight) from the Basketball Reference player index pages.²

Combining these two sources is the data-integration element of the project: the height and weight attributes do not exist in the per-game tables and had to be scraped separately and joined on the player name.

B. Dataset Overview

TABLE I
DATASET OVERVIEW

Property	Value
Primary source	Basketball Reference
Secondary source	Basketball Reference player index
Timeframe	1975 – 2025
Raw rows	26,292
Raw columns	32
Modeling rows	15,020
Modeling features	39
Task type	Binary classification

The raw per-game table contains standard season-average statistics, including Games Played (G), Games Started (GS), Minutes Played (MP), field-goal counts and percentages (FG, FGA, FG%), three-point counts and percentages (3P, 3PA, 3P%), two-point counts and percentages (2P, 2PA, 2P%), effective field-goal percentage (eFG%), offensive, defensive and total rebounds (ORB, DRB, TRB), assists (AST), steals (STL), blocks (BLK), turnovers (TOV), personal fouls (PF), and points per game (PTS). Free-throw columns (FT, FTA, FT%) are present in the raw data but are deliberately excluded from the feature set, as explained in Section V-E.

IV. PROJECT WORKFLOW

The project followed a structured machine learning pipeline:

- 1) Web scraping of NBA per-game statistics and player biographical data.
- 2) Data cleaning and preprocessing.
- 3) Target label construction.
- 4) Feature engineering and integration of the external bio data.
- 5) Feature scaling for distance- and margin-based models.
- 6) Model training with default-configuration estimators.
- 7) Model evaluation and comparison across five algorithms.
- 8) Deployment of an interactive Streamlit web application.

V. DATA PREPROCESSING

Preprocessing was performed in a dedicated notebook and is fully reproducible. Table II summarizes how the row count evolves through the cleaning pipeline.

TABLE II
ROW COUNT THROUGH THE PREPROCESSING PIPELINE

Stage	Rows
Raw scraped data	26,292
After dropping unused columns	26,292
After traded-player de-duplication	21,276
After removing invalid position rows	21,275
After free-throw sample filter ($FTA \geq 1$)	15,020

A. Handling Missing Values and Unused Columns

The Awards column was empty for more than 23,000 of the 26,292 rows and was dropped entirely. The Rk column is only a display row index and was also dropped. Early seasons (1975–1979) predate the three-point line, so 3P, 3PA, and 3P% are missing for those players; these were filled with 0, which is the correct value because those players genuinely attempted no three-pointers. Games Started (GS) is unrecorded before 1982 and was likewise filled with 0. After these steps the dataset contained no missing values.

B. Duplicate Player-Season Handling

When a player is traded mid-season, Basketball Reference lists one row per team plus a combined total row (labeled TOT, 2TM, or 3TM). For each (season, player) pair, only the combined row was kept, so each player-season is represented exactly once. This reduced the data from 26,292 to 21,276 rows. A single row with an invalid position label was then removed.

C. Free-Throw Sample Filter

Only player-seasons with at least one free-throw attempt per game ($FTA \geq 1.0$) were retained. Players with very few attempts have an unstable FT% — a player who takes two free throws all season and makes both has a FT% of 100%, which is not a meaningful measure of skill. This filter ensures every retained row has a stable, interpretable free-throw percentage and left 15,020 modeling rows.

D. Target Label Construction

The target is a binary label defined by a fixed threshold on season free-throw percentage:

$$\text{Target} = \begin{cases} 1, & \text{if } FT\% \geq 0.75 \quad (\text{Good}) \\ 0, & \text{if } FT\% < 0.75 \quad (\text{Below-Average}) \end{cases} \quad (1)$$

The value 0.75 was chosen deliberately for four reasons:

- It is the universally recognized NBA free-throw benchmark and is therefore immediately interpretable to any reader or analyst.
- It is defined *independently of the dataset*. A threshold based on the league-average FT% would make every row’s label depend on aggregate statistics of other rows; a fixed constant does not, which is methodologically cleaner.
- It is effectively era-independent. The NBA league-average FT% moved only between 0.728 and 0.784 across

¹<https://www.basketball-reference.com/leagues/>

²<https://www.basketball-reference.com>

all 50 seasons studied; measured label agreement between a fixed-0.75 target and a per-season league-average target is 94.8%, so the two definitions are nearly equivalent and the simpler one is preferred.

- The median FT% of the filtered data is 0.758, so the 0.75 threshold produces a well-balanced split.

After labeling, the dataset contains 8,151 good shooters (54.3%) and 6,869 below-average shooters (45.7%), shown in Fig. 1. This near-balanced distribution reduces classification bias and makes accuracy a meaningful headline metric without resampling.

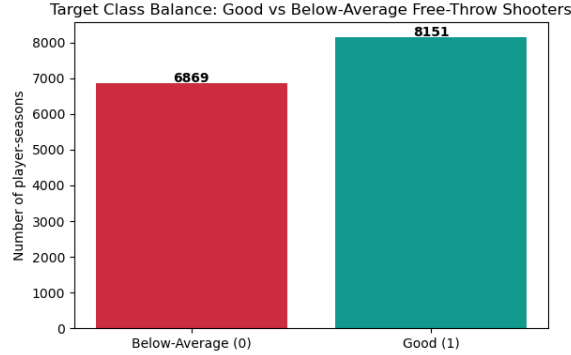


Fig. 1. Target class balance: good versus below-average free-throw shooters.

E. Data Leakage Prevention

The single most important design decision in this project is the explicit prevention of *data leakage*. The columns FT (free throws made), FTA (free-throw attempts), and FT% (free-throw percentage) are **never** used as model features:

- FT% is the direct source of the target label.
- FT and FTA together allow FT% to be reconstructed exactly, since $FT\% = FT / FTA$.

Including any of these would let a model re-derive the target from its own definition, producing an artificially high accuracy that would not generalize to genuine prediction. These columns are dropped from the feature set in the preprocessing notebook.

F. Historical Free-Throw Features

Although the current season’s free-throw statistics are excluded, a player’s free-throw record from *prior* seasons is both legitimate and highly informative. Three historical features were engineered using only data observed before the season being predicted:

- `career_ft_pct`: cumulative FT% over all of the player’s previous seasons.
- `prev_ft_pct`: FT% in the immediately preceding season.
- `career_seasons`: number of prior seasons played (0 for a rookie).

These are computed with chronologically ordered cumulative operations that exclude the current row, so no future information leaks backward. For rookie seasons, where no

history exists, `career_ft_pct` and `prev_ft_pct` were filled with the global mean FT% of 0.7414, a neutral prior for an unobserved player.

G. Integration of External Bio Data

Player height and weight, scraped separately, were merged into the dataset on player name, successfully matching 14,530 of the 15,020 player-seasons. The small number of unmatched rows were filled with the median height and weight. These two attributes (`height_in`, `weight_lb`) constitute the multi-source integration component of the project.

H. Feature Engineering

Six ratio features were engineered to capture playing style and efficiency beyond raw counting statistics:

- $ast_to_tov = AST / TOV$ — ball-handling efficiency.
- $pts_per_fga = PTS / FGA$ — scoring efficiency per shot.
- $threep_rate = 3PA / FGA$ — reliance on three-point shooting.
- $reb_per_min = TRB / MP$ — rebounding intensity.
- $stocks_per_min = (STL + BLK) / MP$ — defensive activity.
- $scoring_load = FGA / MP$ — shooting volume per minute.

Division by zero was handled by treating a zero denominator as a ratio of zero. The categorical `Pos` field was reduced to a primary position and one-hot encoded into five indicator columns (`Pos_C`, `Pos_PF`, `Pos_PG`, `Pos_SF`, `Pos_SG`). The final modeling matrix contains 39 features, listed by group in Table III.

TABLE III
FEATURE GROUPS (39 FEATURES TOTAL)

Group	Features
Playing time	Age, G, GS, MP
Shooting	FG, FGA, FG%, 3P, 3PA, 3P%, 2P, 2PA, 2P%, eFG%
Counting stats	ORB, DRB, TRB, AST, STL, BLK, TOV, PF, PTS
Historical FT	<code>career_ft_pct</code> , <code>prev_ft_pct</code> , <code>career_seasons</code>
Physical (bio)	<code>height_in</code> , <code>weight_lb</code>
Engineered ratios	<code>ast_to_tov</code> , <code>pts_per_fga</code> , <code>threep_rate</code> , <code>reb_per_min</code> , <code>stocks_per_min</code> , <code>scoring_load</code>
Position (one-hot)	<code>Pos_C</code> , <code>Pos_PF</code> , <code>Pos_PG</code> , <code>Pos_SF</code> , <code>Pos_SG</code>

I. Train-Test Split

The dataset was divided into 80% training data (12,016 rows) and 20% test data (3,004 rows) using a fixed random seed (`random_state = 42`). The identical split is reproduced in every model notebook and in the comparison notebook, so all five algorithms are trained and evaluated on exactly the same data.

J. Feature Scaling

Feature scaling was applied with `StandardScaler` for the three models sensitive to feature magnitude — KNN, SVC, and Logistic Regression — while the tree-based models

(Random Forest and Gradient Boosting) were trained on raw, unscaled features, as tree splits are invariant to monotonic feature transformations. To avoid leakage from the test set, each scaler was fit on the training set only and then applied to the test set. The standardization transform is:

$$Z = \frac{X - \mu}{\sigma} \quad (2)$$

where X is the original value, μ the training-set mean, and σ the training-set standard deviation. All preprocessing steps were implemented procedurally, without scikit-learn Pipeline objects, so that every transformation is explicit and inspectable.

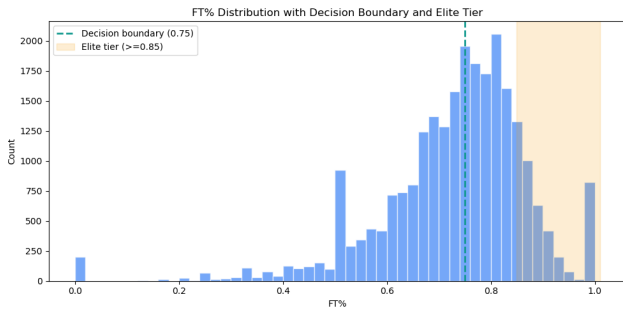


Fig. 2. Distribution of season FT% across all players. The dashed line marks the 0.75 decision boundary; the shaded region marks the elite tier ($FT\% \geq 0.85$).

VI. EXPLORATORY DATA ANALYSIS

Exploratory analysis was carried out before modeling to understand the data and to guide feature engineering.

Fig. 2 shows the distribution of season FT%. The bulk of player-seasons lie between 0.60 and 0.85, with the 0.75 decision boundary cutting through the dense central region — confirming that the chosen threshold separates a genuinely contested population rather than trivially splitting clear extremes.

Fig. 3 ranks every numeric feature by its Pearson correlation with the target. The strongest *positive* predictors are three-point activity (3PA, 3P, *threep_rate*, 3P%), overall scoring (PTS, FGA), and the historical free-throw features (*prev_ft_pct*, *career_ft_pct*). The strongest *negative* predictors are interior-play indicators: *reb_per_min*, *height*, *weight*, *blocks*, and offensive rebounds. This matches basketball intuition — perimeter shooters tend to be reliable free-throw shooters, while big men generally are not.

Fig. 4 plots the NBA league-average FT% over time. It stays within the narrow 0.728–0.784 band across five decades, which empirically justifies treating the fixed 0.75 threshold as era-independent.

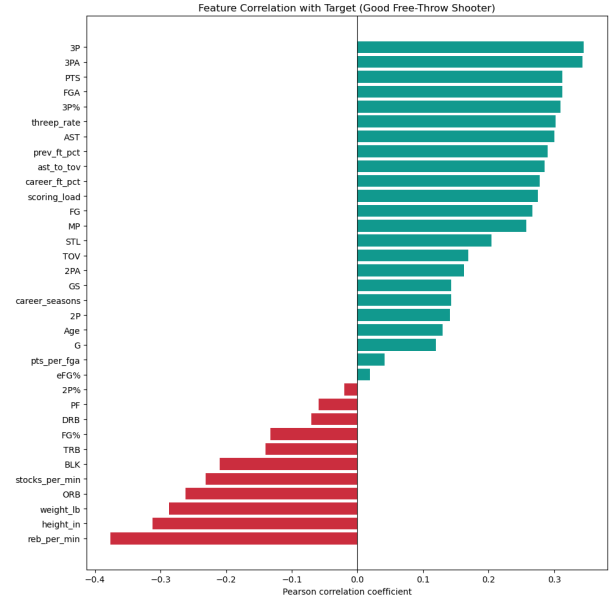


Fig. 3. Pearson correlation of each feature with the target. Green bars are positive predictors, red bars negative.

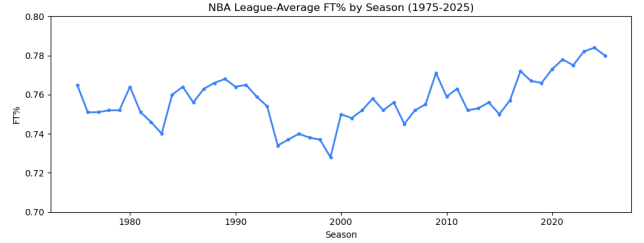


Fig. 4. NBA league-average FT% by season (1975–2025). The value stays within a narrow band, supporting an era-independent fixed threshold.

VII. MACHINE LEARNING MODELS

Five classification algorithms were implemented, one as the primary responsibility of each team member, all on the identical dataset and train-test split. Each model was trained with its default configuration using fixed constructor settings. Table IV lists the configuration used for each model.

A. *K-Nearest Neighbors*

KNN classifies a player-season by majority vote among its k most similar training examples in the standardized feature space. It is non-parametric and makes no assumption about the form of the decision boundary, but it is sensitive to feature scaling and to the dimensionality of the space. The model uses $k = 5$ neighbors.

B. *Support Vector Classifier*

SVC finds the decision boundary that maximizes the margin between the two classes. An RBF kernel is used so that the model can represent non-linear boundaries, with default scikit-learn regularization and gamma settings.

C. Logistic Regression

Logistic Regression models the log-odds of the positive class as a linear combination of the features. It is fast, stable, and highly interpretable, and serves as a strong linear baseline against the non-linear models. The model uses the `liblinear` solver with a maximum of 1,000 iterations.

D. Random Forest

Random Forest trains many decision trees on bootstrapped samples with random feature subsets and averages their votes. This reduces the variance of individual trees and captures non-linear feature interactions. The model uses 100 trees with default depth settings.

E. Gradient Boosting

Gradient Boosting builds an ensemble of shallow decision trees sequentially, where each new tree corrects the errors of the current ensemble. The model uses 100 estimators, a learning rate of 0.1, and a maximum depth of 3.

TABLE IV
MODEL CONFIGURATIONS

Model	Key constructor arguments
KNN	<code>n_neighbors = 5</code>
SVC	<code>kernel = rbf</code> ; default <code>C</code> and <code>gamma</code>
Logistic Regression	<code>solver = liblinear</code> ; <code>max_iter = 1000</code>
Random Forest	<code>n_estimators = 100</code> ; <code>random_state = 42</code>
Gradient Boosting	<code>n_estimators = 100</code> ; <code>learning_rate = 0.1</code> ; <code>max_depth = 3</code> ; <code>random_state = 42</code>

VIII. EVALUATION METRICS

Because the two classes are nearly balanced, accuracy is a meaningful headline metric, but it is reported together with precision, recall, F1-score, and the confusion matrix to give a complete picture of each model's behavior. The metrics are defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (4)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (5)$$

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (6)$$

where TP , TN , FP , and FN are the counts of true positives, true negatives, false positives, and false negatives. Precision, recall, and F1 are reported for the positive class (good shooter). All evaluation uses the same held-out 20% test set of 3,004 player-seasons.

IX. EXPERIMENTAL RESULTS

Table V reports the test-set performance of all five models, ordered by accuracy. Gradient Boosting achieved the best accuracy at 78.06%, ahead of SVC at 77.23% and Random Forest at 76.96%. Logistic Regression and KNN trailed at 74.23% and 71.64% respectively. The accuracy comparison is visualized in Fig. 5.

TABLE V
MODEL PERFORMANCE ON THE TEST SET (%)

Model	Accuracy	Precision	Recall	F1
Gradient Boosting	78.06	79.65	79.70	79.68
SVC	77.23	78.31	79.95	79.12
Random Forest	76.96	79.09	77.91	78.50
Logistic Regression	74.23	75.25	77.85	76.53
KNN	71.64	73.35	74.52	73.93

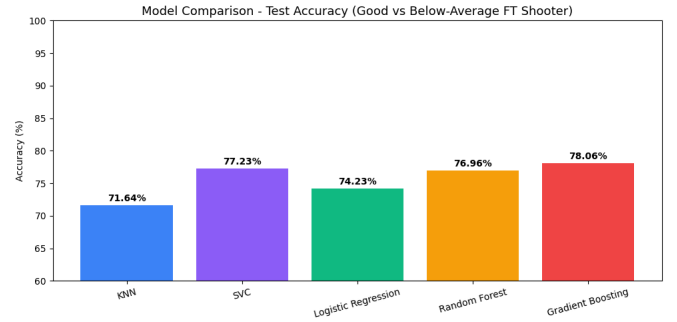


Fig. 5. Test accuracy of the five classification models.

The confusion matrices in Fig. 6 through Fig. 10 show how each model distributes its predictions across the two classes. The strongest models are roughly symmetric, making a similar number of errors on each class, which is desirable given the balanced target.

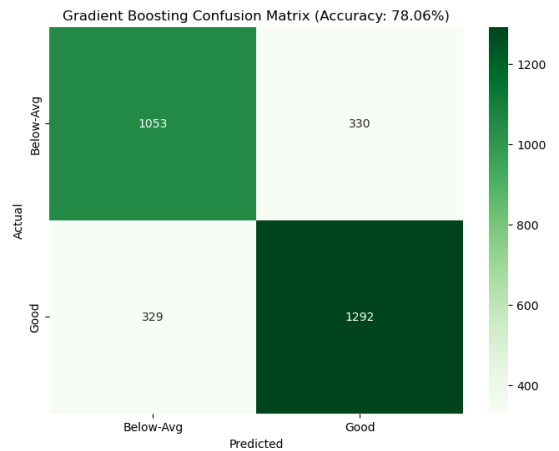


Fig. 6. Gradient Boosting confusion matrix.

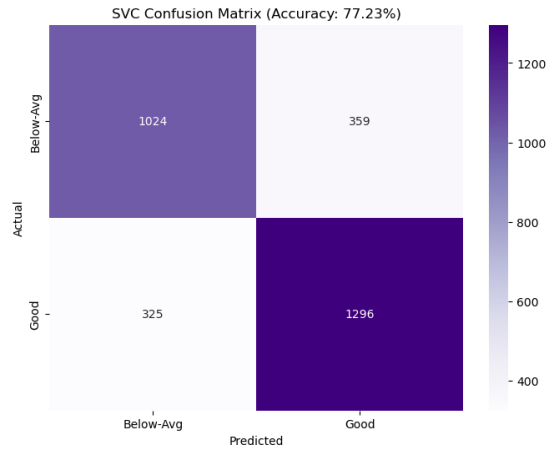


Fig. 7. SVC confusion matrix.

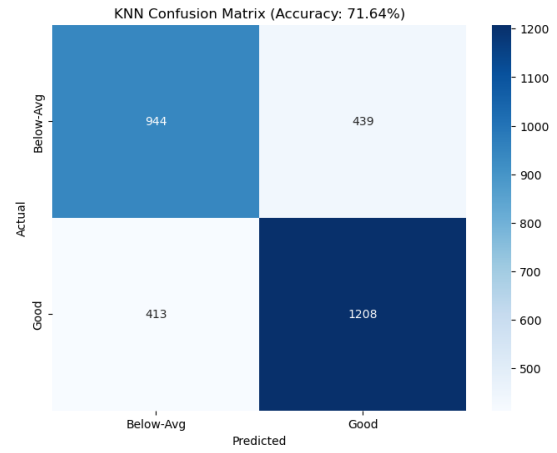


Fig. 10. KNN confusion matrix.

X. DISCUSSION

A. Why the Models Cluster in the 73–78% Range

All five models fall within a 71–78% accuracy band. This is an honest and expected result rather than a weakness. Because the fixed 0.75 threshold classifies *every* player-season that has a meaningful free-throw sample — nobody is excluded — the models must label the genuinely uncertain players who shoot close to 75%. The 71–78% range therefore reflects what machine learning can realistically achieve on the full population, including the hard middle cases, rather than an inflated figure obtained by only testing on clear extremes.

B. Comparison and Trade-offs

The non-linear models (Gradient Boosting, SVC, Random Forest) outperformed Logistic Regression, indicating that the relationship between box-score statistics and free-throw skill is not fully linearly separable. Gradient Boosting and SVC were essentially tied at the top; Gradient Boosting is preferable in deployment because it does not require feature scaling and exposes feature importances, while SVC is more sensitive to its hyperparameters and scales less well to larger data. Random Forest performed slightly below the other two ensembles but remains competitive and is the most robust to overfitting with minimal tuning.

KNN produced the weakest result. In a 39-dimensional feature space, distance-based neighbor voting suffers from the curse of dimensionality: distances become less discriminative, so even $k = 5$ neighbors cannot match the structured models. Logistic Regression, although linear, still produced a respectable 74.23% and is the most interpretable model, making it a reasonable lightweight baseline.

In summary, there is a clear trade-off: Logistic Regression offers interpretability and speed, KNN offers conceptual simplicity but poor high-dimensional behavior, and the ensemble and kernel methods offer the best accuracy at the cost of interpretability and tuning effort.

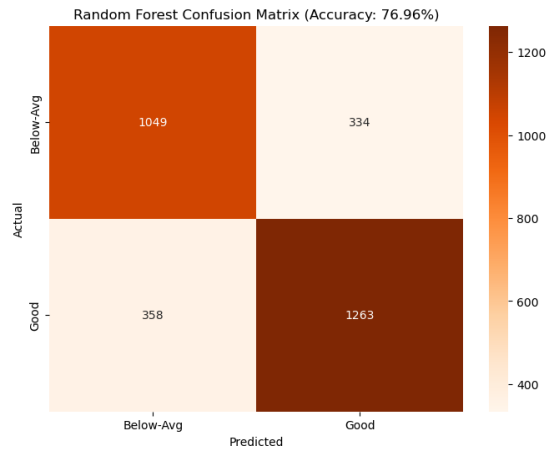


Fig. 8. Random Forest confusion matrix.

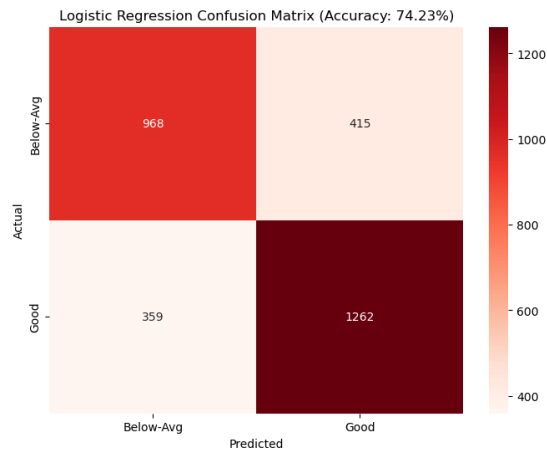


Fig. 9. Logistic Regression confusion matrix.

C. What Drives the Predictions

The exploratory correlation analysis (Fig. 3) and the model behavior agree: the historical free-throw features (`career_ft_pct`, `prev_ft_pct`) and perimeter-shooting features (`3PA`, `3P`, `threept_rate`) are the strongest signals. This is intuitive — free-throw shooting is a stable individual skill, so a player’s own past record is the best available evidence, and perimeter shooting and free-throw shooting share an underlying shooting touch.

D. Role of the External Bio Data

The height and weight features form the multi-source data-integration component of the project. Their measured contribution to predictive accuracy is marginal, because height and weight are already partially reflected in position and rebounding statistics. They were nonetheless retained, both because they are the project’s genuine data-integration element and because reporting their limited impact honestly is itself a point of methodological rigor.

E. Alternative Threshold Framing

An alternative, stricter design would classify only *elite* shooters ($FT\% \geq 0.85$) against *weak* shooters ($FT\% < 0.65$), discarding the ambiguous middle group. That framing can reach a higher headline accuracy of roughly 92%, but only because it removes the roughly 47% of players who are hardest to classify — the model is never asked to make a difficult call. The fixed 0.75 threshold was chosen as the primary target precisely because it classifies every player-season and therefore produces an honest estimate of real-world performance.

XI. PHASE 2: STREAMLIT DEPLOYMENT

To make the trained models usable in practice, the system was deployed as an interactive web application built with Streamlit. The application loads the five trained models, their scalers, and the saved ordered feature list, and provides three pages reachable from a sidebar menu:

- **Home** — a project summary with key dataset statistics and the best-performing model.
- **Prediction** — an input form, organized into free-throw history, player information, shooting statistics, and performance groups, where the user enters statistics and selects a model.
- **Model Comparison** — the results table, the accuracy bar chart, and all five confusion matrices.

On the prediction page, the application reconstructs the full 39-feature vector from the user’s inputs: it computes the engineered ratio features, derives the made-shot counts, one-hot encodes the position, and reindexes the row to the exact training feature order stored in `feature_columns.pkl`. The appropriate scaler is then applied for KNN, SVC, and Logistic Regression, while Random Forest and Gradient Boosting receive the raw vector. The selected model returns a real-time classification of the player as a good or below-average free-throw shooter. Fig. 11 and Fig. 12 show the interface.

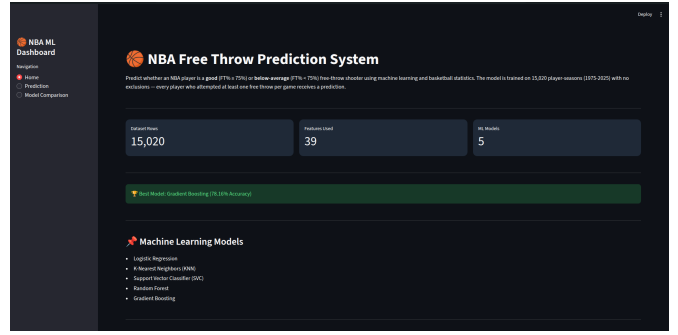


Fig. 11. Streamlit application — Home page.

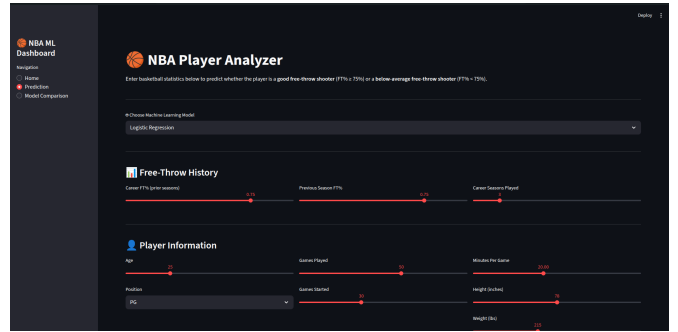


Fig. 12. Streamlit application — Prediction page.

XII. LIMITATIONS

Several limitations were identified during the project:

- Season-average statistics hide game-to-game consistency: two players with the same FT% may differ greatly in reliability.
- Psychological and situational factors — clutch pressure, fatigue, crowd effects — are not represented in box-score data.
- Injury history and changes in shooting mechanics across a career are not captured.
- Coaching, training regimens, and practice habits are unavailable.
- NBA playstyles and rules changed substantially between 1975 and 2025, introducing era-related variation into the features.
- The external bio merge could not match roughly 490 player-seasons by name; these were filled with median values.

XIII. CONCLUSION

This project presented a complete machine learning pipeline for classifying NBA player-seasons as good or below-average free-throw shooters. An original dataset of 26,292 player-season records was assembled by web scraping two Basketball Reference sources and integrating them. The data underwent careful preprocessing, with particular emphasis on preventing data leakage by excluding all current-season free-throw quantities from the feature set, yielding 15,020 modeling rows described by 39 features.

Five classification algorithms were trained with default-configuration estimators and compared on an identical test set. Gradient Boosting achieved the best test accuracy at 78.06%, closely followed by SVC and Random Forest. The 71–78% range across all models is an honest reflection of the difficulty of the task once every player — including the genuinely uncertain cases — must be classified. Finally, the system was deployed as an interactive Streamlit web application that delivers real-time predictions and model comparison. Overall, the project demonstrates both the value and the realistic limits of machine learning in sports analytics, and the importance of leakage-aware experimental design.

XIV. FUTURE WORK

Future improvements may include incorporating advanced and tracking-based NBA statistics, modeling game-level rather than season-average data to capture consistency, adding more external data sources such as college or international shooting records, experimenting with deep learning architectures, integrating a live NBA data API for automatic updates, and deploying the dashboard to a public cloud platform.

REFERENCES

- [1] Basketball Reference, Available: <https://www.basketball-reference.com>
- [2] Basketball Reference Leagues, Available: <https://www.basketball-reference.com/leagues/>
- [3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. Available: <https://scikit-learn.org>
- [4] Streamlit Documentation, Available: <https://streamlit.io>
- [5] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed., O’Reilly Media, 2022.
- [6] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.
- [7] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [8] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.